

Práctica 1: Analizadores léxicos con Lex (Flex)

Santiago Zapata Amezcua

Compiladores 2027-1

Facultad de Ciencias, UNAM

Profesora: Ariel Adara Mercado Martínez

1. Primer programa en Flex

1.1. Ejercicio 1

¿Qué ocurre si en la primera sección se quitan las llaves al nombre de la macro letra? Flex ya no lo toma como la macro, sino como texto literal. La definición queda así:

```
palabra letra+
```

Esto significa “letr” seguido de una o más “a”. El programa compila sin error, pero la macro palabra solo reconoce cadenas como letra, letraa, letraaa, etc. Una palabra normal como hola ya no entra en ninguna regla y flex solo la imprime tal cual.

1.2. Ejercicio 2

¿Qué ocurre si en la segunda sección se quitan las llaves a las macros?

Pasa lo mismo que en el ejercicio anterior: las reglas se vuelven cadenas literales. La regla digito+ reconoce “digit” seguido de una o más “o”, la regla palabra solo reconoce la palabra “palabra” y la regla espacio solo reconoce la palabra “espacio”. Compila, pero ya no reconoce números, palabras ni espacios de verdad, y todo lo demás se imprime tal cual.

1.3. Ejercicio 3

¿Cómo se escribe un comentario en flex?

Con /* comentario */, igual que en C. En la sección de declaraciones se puede poner al inicio de la línea. En la sección de reglas hay que dejarle al menos un espacio al inicio o ponerlo dentro de las llaves de la acción léxica, porque si empieza en la primera columna flex lo toma como una expresión regular. En la sección de código de usuario se usan los comentarios normales de C++ (// y /* */).

1.4. Ejercicio 4

¿Qué se guarda en yytext?

Se guarda el lexema, es decir, el texto de la entrada que acaba de coincidir con la expresión regular. Es un apuntador a char terminado en carácter nulo. Su longitud se guarda en yyleng, y desde fuera de la clase se obtienen con los métodos YYText() y YYLeng().

1.5. Ejercicio 5

¿Qué pasa al ejecutar el programa e introducir cadenas de caracteres y de dígitos por la consola?

El programa se queda esperando entrada y cada vez que se presiona Enter analiza la línea. Por cada grupo de dígitos imprime “Encontré un número:” seguido del número, y por cada grupo de letras imprime “Encontré una palabra:” seguido de la palabra. Los espacios y saltos de línea se ignoran. Si se escribe algo mezclado como abc123, lo separa en la palabra abc y el número 123. El programa termina con Ctrl+D, que manda el fin de archivo.

1.6. Ejercicio 6

¿Qué ocurre si introducimos caracteres como “*” en la consola?

Como el asterisco no coincide con ninguna regla, flex usa su regla por defecto, que es hacer ECHO, o sea, imprimir el carácter tal cual. Por ejemplo, con la entrada hola*mundo se imprime “Encontré una palabra: hola”, luego el asterisco pegado a la siguiente línea y luego “Encontré una palabra: mundo”. No marca ningún error, simplemente lo repite.

1.7. Ejercicio 7

Modificar el código anterior en un archivo nuevo para que reconozca hexadecimales de C++, 5 palabras reservadas de C++, identificadores válidos de C++ con longitud máxima de 32 caracteres y los espacios en blanco.

El código está en el archivo ejercicio7.ll dentro de la carpeta Primer programa en Lex. Las expresiones regulares que usé son las siguientes:

```
digito      [0-9]
hexdigito   [0-9a-fA-F]
letra       [a-zA-Z]
sufijo      ([uU](l|L|ll|LL)?|(1|L|ll|LL)[uU]?)
hexadecimal 0[xX]{hexdigito}+{sufijo}?
reservada   if|else|while|return|class
id          ({letra}|_){letra}{digito}|_{0,31}
idlargo     ({letra}|_){letra}{digito}|_{32,}
espacio     [ \t\n]+
```

Los hexadecimales empiezan con 0x o 0X, siguen uno o más dígitos hexadecimales y al final pueden llevar un sufijo como u, l, ll o combinaciones como ul. Las palabras reservadas que elegí son if, else, while, return y class. Los identificadores empiezan con letra o guion

bajo y después pueden llevar letras, dígitos o guiones bajos, con un máximo de 32 caracteres en total gracias al operador `{0,31}`. Si un identificador mide más de 32, la regla `idlargo` lo reconoce y lo reporta como demasiado largo. Los espacios en blanco son espacios, tabuladores y saltos de línea.

La regla de palabras reservadas va antes que la de identificadores, así cuando las dos reconocen la misma cadena gana la palabra reservada.

Se compila igual que el primer programa: `flex++ ejercicio7.ll`, luego `g++ lex.yy.cc -o ejercicio7` y se ejecuta con `./ejercicio7`.

2. Analizador léxico para el lenguaje C_1

2.1. Ejercicio 8

Describir el conjunto de terminales y la expresión regular que reconoce a cada uno en `lexer.ll`.

Los terminales del lenguaje C_1 son los paréntesis, la coma, el punto y coma, los identificadores, las palabras reservadas `if`, `int`, `while`, `else` y `float`, y los números. En la siguiente tabla está cada uno con su número de token y su expresión regular.

Token	Número	Expresión regular
LPAR	6	"("
RPAR	7	")"
COMA	8	","
PYC	9	;"
ID	10	(<code>{letra} _</code>)(<code>{letra} {digito} _</code>)*
IF	11	"if"
INT	12	"int"
WHILE	13	"while"
ELSE	14	"else"
FLOAT	15	"float"
NUMERO	16	<code>{entero} {decimal}{exponente}? {entero}{exponente}</code>

Tabla 1: Terminales del lenguaje C_1

Las macros que se usan en la tabla son:

```

digito    [0-9]
letra     [a-zA-Z]
entero    {digito}+
decimal   ({digito}*"."{digito}+|{digito}+ "."{digito}*)
exponente [eE] [+]?{digito}+

```

Un identificador empieza con letra o guion bajo y puede seguir con letras, dígitos o guiones bajos. Un número puede ser entero (12345), decimal (1.2, .5 o 10.) y cualquiera de los dos puede llevar exponente (1.2e6, 7E-3).

Además, los espacios en blanco se reconocen con `[\t\n\r]+` y se ignoran, y cualquier otro carácter se reporta como error léxico junto con su número de línea.

Las palabras reservadas van antes que ID en `lexer.ll` para que, cuando empatan en longitud, gane la palabra reservada. Por eso `if` se reconoce como IF, pero `if3` se reconoce como ID, ya que flex siempre toma la coincidencia más larga.

2.2. Ejercicio 9

Generar acciones léxicas para cada terminal del lenguaje en `Lexer.cpp`, de modo que se muestre en pantalla la salida esperada con el archivo prueba.

Las acciones léxicas están escritas en `lexer.ll` y `flex++` las pasa a `Lexer.cpp` cuando lo genera, porque en `lexer.ll` se usa la opción `outfile="Lexer.cpp"`. Cada acción regresa el número de su token, que está definido en `tokens.hpp`. Después, `main.cpp` abre el archivo de entrada, le va pidiendo tokens al analizador hasta llegar al fin del archivo e imprime cada uno como el número del token, una coma y el lexema. Con el archivo prueba se obtiene exactamente la salida esperada.

2.3. Ejercicio 10

Crear un Makefile.

El Makefile está en la carpeta `C_1`. Con `make` se genera `Lexer.cpp` a partir de `lexer.ll` y se compila el ejecutable `compiler`. Con `make run` se ejecuta el analizador con el archivo prueba, y con `make clean` se borran `Lexer.cpp` y el ejecutable.

3. Extras

3.1. Ejercicio 11

Documentar el código.

Todos los archivos de código tienen comentarios que explican qué hace cada parte.

3.2. Ejercicio 12

Proponer 4 archivos de prueba nuevos, 2 válidos y 2 inválidos.

Los archivos están en la carpeta `C_1` y se llaman `prueba_valida1`, `prueba_valida2`, `prueba_invalida1` y `prueba_invalida2`. Se ejecutan igual que el archivo prueba, por ejemplo `./compiler prueba_valida1`.

Los archivos válidos solo usan elementos del lenguaje `C_1`: palabras reservadas, identificadores, números, paréntesis, comas y punto y coma. En `prueba_valida1` hay declaraciones de funciones y en `prueba_valida2` hay números en distintos formatos, como 3.1416, .5, 10., 6.022e23 y 7E-3. En ambos casos el analizador imprime todos los tokens sin errores.

Los archivos inválidos tienen caracteres que no pertenecen al lenguaje, como `=`, `+`, `$`, `#`, `>`, `{`, `}` y `@`. El analizador marca cada uno de estos caracteres como error léxico con su número de línea y sigue analizando el resto del archivo.